Backup og gendannelse

øvet, ikke forsikret

Målt 9. november 2026 · steamgears.eu · S5.8

Roadmappet krævede »RPO/RTO skrevet ned, og én gendannelse ført hele vejen til staging med tidtagning«. Det sidste kan ikke gøres endnu — der findes ingen staging-side (S5.1, ejeren). Det første kan, og det er nu et **måltal i stedet for en forsikring**: dump, gendannelse, rækkeparitet for hver tabel og oprydning, kørt mod den levende database.

Værktøjet er tools/wp_db_drill.py — ikke et dokument med en opskrift.

1. Hvad der blev målt

Måltal	Resultat	Metode
Dump	3 338 987 bytes gzip, 5 s, sha256 e1d028e8…	mysqldump --single-transaction --quick --skip-lock-tables
Rå størrelse	64,4 MB / 269 tabeller (WP-prefix clk857797e3d3_)	information_schema
Gendannelse (RTO)	6 s	afspillet i samme database under prefixet sgtest_
Paritet	269 af 269 tabeller med identisk rækkeantal, 0 mangler	én UNION ALL-tælling pr. side, sammenlignet
Oprydning	269 tabeller + 4 visninger droppet, 0 tilbage	DROP TABLE + DROP VIEW
RPO	0 t ved kommando (værtens daglige backup: højst 24 t)	dump tages on-demand

Kørslen tog **20 s** i alt inklusive oprydning, og den efterlod hverken tabeller eller filer.

2. De tre ting der gør en naiv gendannelse farlig

2.1 Der kan ikke oprettes en ekstra database

`SHOW GRANTS` på forbindelsen: `GRANT ALL PRIVILEGES ON steamgears\_eu\_db.*` — intet på `*.*`. Et forsøg på `CREATE DATABASE sgbk_drill_tmp` svarer `ERROR 1044 (42000)`: «Access denied». Gendannelsen skal altså ske i **samme database**, og hver oprettet tabel skal ryddes bagefter. Værktøjet fejler hvis én står tilbage.

2.2 Et gammelt WordPress-prefix ligger stadig i databasen

Databasen bærer to prefixes:

Prefix	Tabeller	Rækker (udvalg)	Status
clk857797e3d3_	269 + 4 visninger	posts 1 360 · postmeta 15 065	det levende site
clk4219fc96b5_	12	posts 4 · postmeta 2 · options 128 · users 1	forældet rest fra en tidligere installation

`mysqldump` tager dem alle med, og den skriver `DROP TABLE IF EXISTS` før hver tabel. En gendannelse af et prefiks-bytte ville derfor **slette og genskabe de 12 gamle tabeller** med dumpets indhold — usynligt, fordi de ikke bruges af sitet, men et tab af alt hvad der var skrevet i dem efter dumpet. Værktøjet udelader dem på dump-siden (`--ignore-table` pr. tabel), så en øvelse aldrig kan røre dem.

Om de 12 tabeller skal slettes, er **ejerens beslutning** — de er små (136 rækker) og harmløse, men de er også den slags der gør en gendannelse sværere at ræsonnere om. Spørgsmålet står i

roadmappet.

2.3 Visninger tælles som tabeller

`information_schema.tables` indeholder både `BASE TABLE` og `VIEW`. Fire af databasens objekter er visninger (`*_uap_*_view`). Første kørsel droppede 273 tabeller og efterlod **4** — `DROP TABLE` fejler på en visning. Rettelsen er at filtrere på `table_type` og bruge `DROP VIEW` for de fire. Et værktøj der kun tæller »tabeller« og kun dropper tabeller, rapporterer derfor et rent gennemløb med skrald tilbage.

3. Sådan bruges den

```
export PATH="$HOME/.novamira-npm/bin:$PATH"
python3 tools/wp_db_drill.py            # kør mod live, skriv reports/db-drill-<dato>.json
python3 tools/wp_db_drill.py --keep     # lad dumpet ligge på serveren (til en rigtig gendannelse)
```

Værktøjet lægger sin egen `my.cnf` (0600) i en midlertidig mappe på serveren, så **hverken brugernavn eller password** står i proceslisten eller i repoet, og rydder mappen bagefter. Det returnerer exit-kode 0 kun når dump, gendannelse, paritet og oprydning alle er rene — og et gennemløb uden paritetstal tæller ikke som bestået.

Rapporten ligger i reports/db-drill-2026-11-09.json.

4. Et engangstal ældes — derfor er øvelsen nu et månedligt værn

Øvelsen er kun et værn, hvis den bliver **kørt igen**. Et tal fra 9/11 siger intet om databasen i december: den vokser, og gendannelsestiden følger med. Derfor er kadencen og tallene nu maskinelle:

- **Historikken** `reports/db-drill-history.json` får **én række pr. kørsel** (dato, klokkeslæt, dumpstørrelse, gendannelsestid, tabeller, afvigelser, rester, dom). Dagsfilen `reports/db-drill-<dato>.json` overskrives af en ny kørsel samme dag — historikken gør ikke, så RTO og paritet bliver en **serie** i stedet for ét tal.
- **Værnet** tools/db_drill_check.py holder to ting op: **fristen** (seneste kørsel må højst være **31 dage** gammel, med **3 dages varsel** i `--due`) og **tallene** (seneste kørsel skal være `ok`, med 0 rækkeafvigelser, 0 manglende tabeller, 0 rester, ikke-tomt dump og **RTO under 300 s** — målt 6 s, så budgettet fanger en regression, ikke en dagsform). 15 offline-fixtures dækker forfalden frist, én afvigelse, rester, for langsom RTO, tomt dump, nul tabeller, korrupte kilder og trenden.
- **Kadencen** måles hver nat: `backup-øvelse-selftest` og `backup-øvelse (frist + paritet)` kører i tools/natlige-vaern.sh, så fristen tælles ned dagligt og ikke årligt. Den **første i hver måned** kører .github/workflows/backup-doelse.yml og skriver trenden i job-summaryen — et varsel før fristen løber ud, ikke efter.

Anden målte kørsel, 9/11 kl. 05:07 (samme dag, ny kørsel — historikken har nu to datapunkter):

Kørsel	Dump	Gendannelse	Tabeller	Afvigelser	Rester
9/11 04:2x	3 338 987 B / 5 s	6 s	269	0	0
9/11 05:07	3 335 364 B / 4 s	6 s	269	0	0

Hele anden kørsel tog **16,5 s** inklusive oprydning. Forskellen i dumpstørrelse (3 538 bytes) er indhold, ikke metode — derfor er det historikken og ikke et enkelt tal, der skal vurderes.

Værnet kan ikke selv udføre øvelsen. Den kræver en databaseforbindelse gennem Novamira-MCP, og det access-token findes kun i keep-alive-kørslen, som **roterer** refresh-tokenet — en fornyelse mere i en anden workflow ville slå de to kørsler ihjel. Derfor er arbejdsdelingen: den natlige kørsel **måler** fristen og fejler når den er overskredet, månedskørslen **varsler** med den præcise kommando, og selve kørslen er en agenthandling (`python3 tools/wp_db_drill.py`) med rapporten committet. Skal den også udføres automatisk, skal der findes en databaseforbindelse uden

om MCP — det er en ejerbeslutning, ikke en kodeopgave.

5. Hvad der stadig mangler

- **Øvelsen ført til staging.** Uden S5.1 findes der ikke et isoleret sted at gendanne til, så beviset er »dumpet kan afspilles i samme database«, ikke »siden kører efter en gendannelse på en anden maskine«.
- **Filernes halvdel.** Kun databasen er dækket. `wp-content/uploads` er 2,4 GB og kræver enten værtens filbackup eller en rsync-vej, som ikke findes herfra.
- **Tid til at flytte dumpet.** 3,3 MB gzip kan hentes gennem MCP, men værktøjet gør det ikke — en rigtig gendannelse på en anden maskine skal først have filen derover.

6. Bilag: hvordan 7.65.5 kom på sitet

Samme tur blev plugin'ets fire patch-versioner lagt på sitet — det kørte 7.65.0, repoet var 7.65.4, så P3.7's og P3.8's besparelser var bygget og grønne i CI uden at være i drift.

Vejen, som den blev målt: zip'en (398 668 bytes, sha256 `e399813b...`) blev sendt gennem MCP i **to base64-dele** à 300 000 tegn, dekodet til en fil i `sys_get_temp_dir()`, verificeret mod sha256 og installeret med `Plugin_Upgrader::install($zip, ['overwrite_package' => true])` — derefter `opcache_reset()` og `w3tc_flush_all()`. Resultat: **7.65.0 → 7.65.4 → 7.65.5**, stadig aktiv — og de fire kontrollerede filer er byte-identiske mellem site og repo.

Målt live bagefter: `SGAP_Admin::dashboard()` giver **17 566 bytes HTML med 0 inline <script> og 0 inline <style>** (mod 40 452 tegn inline CSS før 7.65.2), syv side-scripts i `assets/pages/` og to CSS-filer. `/`, `/shop/`, produktside og `/cart/` svarer **200** (75 124 / 269 514 / 282 839 / 378 718 bytes), og `/wp-json/sgap/v1/tick` svarer **200**.

Fælden: at rendere en admin-side gennem `execute-php` kræver `class-wp-screen.php` + `screen.php` + `template.php` og `wp_set_current_user(1)` — uden dem fejler `set_current_screen()` med `Class "WP_Screen" not found`, og uden `template.php` mangler admin-hjælperne. Klassens navn er `SGAP_Admin`, ikke `SGAP_Core`, selv om filen hedder `core.php`.